

Q1) Deadlock Debugging

Cause of Deadlock

1. P1 locks R1 and waits for R2.
2. P2 locks R2 and waits for R1.
3. Both Processes wait indefinitely, creating a circular wait.

Deadlock Conditions

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

All four conditions are present.

Solution

Use resource ordering:

Both processes should acquire resource in the same order ($R1 \rightarrow R2$).

Conclusion

Deadlock occurs due to circular waiting. A fixed resource acquisition order prevents deadlock.

Q2) CPU Utilisation Problem

Cause

The scheduler log shows:

Ready → Waiting → Ready → Waiting

This indicates processes are frequently waiting for I/O operations.

Effects

1. CPU remains idle
2. Low CPU utilisation (30%).
3. Reduced throughput

Optimisation:

1. Increase multiprogramming
2. Use asynchronous I/O
3. Improve scheduling algorithms
4. Use caching and buffering

Conclusion

Frequent I/O wait cause poor CPU utilisation. Better scheduling and I/O management improve performance.

Q3)

Memory Bottleneck Debugging

Memory Usage

Code = 5MB

Heap = 150MB

Stack $\Rightarrow 25 \times 1\text{MB} = 25\text{MB}$

Total = 180MB

Cause

Frequent page fault occur because:

1. Multiple threads access memory simultaneously.
2. Poor memory locality
3. Heap fragmentation

Optimisation

Reduce thread stack size

Improve memory locality using memory pools and contiguous allocation

Starvation Debugging

Cause

1. Process P5 in Q3 (lowest Priority)
2. Higher-priority processes in Q1 and Q2 continuously receive CPU time

Result

1. P5 waits indefinitely
2. This condition is called starvation.

Optimisation

Use Aging:

- Increase the priority of processes that wait for a long time

Benefits

1. Prevent starvation
2. Ensure
3. Increase Priority

Q5) Context - Switch Overhead Debugging

Given:

Quantum = 1ms

Burst time = 2ms

High I/O frequency

Cause: Threads are preempted before finishing, causing excessive context switches.

Optimisation

Increase time quantum (eg, 4-8ms)

Reduce I/O wait using asynchronous I/O

6) Producer - Consumer

Error: mutex is acquired before empty/full. A thread may block while holding the mutex

Wrong Order

wait(mutex)

wait(empty/full)

Correct Producer

wait(empty)

wait(mutex)

insert(item)

signal(mutex)

signal(full)

Consumer

wait(full)

wait(mutex)

remove(item)

signal(mutex)

signal(empty)

7) Thread Performance

Problem: One thread is created for every request

Effects:

High CPU overhead

Increasing memory usage

Many context switches

Optimisation

Use a thread pool

Use event-driven/asynchronous I/O

Result: Better scalability and stable performance

8) Banker's Algorithm

Need Matrix

Process	Need (A, B, C)
P1	(1, 1, 1)
P2	(1, 1, 1)
P3	(2, 1, 1)

Request by $P_2 = (1, 1, 1)$

Request $\leq$ Need

Request $\leq$ Available

After allocation, a safe sequence exists

$P_2 \rightarrow P_1 \rightarrow P_3$

Conclusion

The request can be granted safely

9) Deadlock Detection and Recovery

Wait-For Cycle

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_1$

Deadlock: Yes, a cycle exists

Detection: Construct a Wait-For Graph and check for cycles

Recovery Methods

Abort one transaction

Resource preemption and rollback

Best choice

Abort one victim transaction because it causes minimum disruption.

10) Monitor-Based Synchronization

Error: Waiting processes are never awakened because `signal()` is missing

Wrong Logic

```
if (busy) wait();
```

Correct Logic

```
while (busy)
```

```
wait(printerfree);
```

```
busy = true;  
// printing  
busy = false;  
signal (printFree);
```

Why:

while rechecks the condition and signal C)
wakes a waiting process

Result: Correct synchronization and fairness

Conclusion:

These answers cover

1. Deadlock
2. CPU Scheduling
3. Memory management
4. Starvation